NOURISH NETWORK

SIH 2026 Team Presentation Guide & Technical Reference Manual

SECTION 1: SIH 2026 PRESENTATION SLIDE UPDATES

Below are the exact updated text blocks for Slides 3, 4, and 6 to ensure your presentation matches your actual codebase and stands up to judge technical Q&A.

Slide 3: Technical Approach (Copy & Paste Ready)

* **Frontend:** HTML5, Vanilla JavaScript (ES6+), Vanilla CSS
* **Backend:** Node.js + Express.js REST API
* **Database/Auth:** PostgreSQL (Supabase Cloud), SQLite3 (WAL Fallback), JWT, bcryptjs, Firebase Auth
* **Architecture Flow:** User Action -> Express REST API -> Database Sync -> Email Dispatch (Google Apps Script / Nodemailer) -> Role Steering -> Vendor/NGO Dashboard

Slide 4: Feasibility and Viability (Copy & Paste Ready)

* **Feasibility:** Lightweight, cost-effective cloud architecture using Node.js, Express.js, PostgreSQL, and SQLite.
* **Feasibility (Scale):** Easy to deploy, maintain, and scale seamlessly based on user demand.
* **Viability (Network):** Creates a sustainable network connecting food businesses, NGOs, and communities.
* **Viability (CSR):** Potential partnerships with corporate CSR initiatives can support operational growth.

Slide 6: Research and References (Copy & Paste Ready)

* **Reference 1:** UNEP Food Waste Index Report -- Food waste and sustainability insights.
* **Reference 2:** FSSAI -- Save Food Share Food Initiative -- Guidance on surplus food redistribution.
* **Reference 3:** FSSAI Surplus Food Guidelines -- Food safety and handling considerations.
* **Reference 4:** Companies Act, 2013 -- Section 135 -- CSR framework.
* **Reference 5:** Google Apps Script, Nodemailer & Firebase Documentation
* **Reference 6:** Node.js, Express.js, PostgreSQL & SQLite Documentation

SECTION 2: BASIC TECHNICAL CONCEPTS (SIMPLE EXPLANATIONS)

Use these everyday analogies when explaining the platform concepts to your teammates or judges.

* **Frontend (The Screen & Buttons):** Everything the user sees and clicks on their screen (HTML layout, CSS styling, JavaScript clicks). Files: index.html, dashboard.html, script.js.
* **Backend (The Hidden Engine):** The server running behind the scenes (server.js using Node.js & Express). It verifies passwords, saves food listings, and sends email alerts.
* **Database (The Digital Filing Cabinet):** The place where all data (user accounts, food listings, claimed orders) is permanently stored so it is not lost when the browser closes.
* **API (The Restaurant Waiter):** An API (Application Programming Interface) carries requests from the frontend to the backend and brings back the response. Analogy: You (Frontend) order food from the Waiter (API), who takes it to the Kitchen (Backend) and brings back your meal.
* **REST API (Standardized Menu):** A REST API follows standard web methods: POST /api/register (create account), POST /api/login (sign in), GET /api/listings (fetch meals), POST /api/orders (claim meal).
* **API Key (VIP Access Pass):** A secret password key that allows your software to talk to third-party services securely (e.g., Google Firebase API keys).

SECTION 3: DEEP-DIVE TECHNICAL Q&A FOR PRESENTATIONS

Q1: Why are there TWO databases (PostgreSQL + SQLite) in your code?

* Local SQLite (database.sqlite): Perfect for local offline testing on your laptop without internet or cloud credentials. Runs in WAL (Write-Ahead Logging) mode for fast concurrent operations.
* Cloud PostgreSQL (Supabase): When deployed on live cloud hosts (Render.com), free servers restart often and wipe local laptop files. PostgreSQL on Supabase keeps all data permanently safe in the cloud.
* In code (database.js): The server checks process.env.DATABASE_URL. If present, it connects to Supabase PostgreSQL; otherwise, it falls back to local SQLite.

Q2: What is a Salt Round? (SALT_ROUNDS = 10)

* Plaintext passwords like "myPass123" are NEVER stored. They are scrambled (hashed) into gibberish using bcryptjs.
* A "Salt" is a random string added to the password before scrambling so two users with "123456" get completely different hashes.
* "10 Salt Rounds" means the algorithm scrambles the password 210 = 1,024 times in a loop! This prevents hacker supercomputers from guessing passwords, while real users log in in 0.05 seconds.

Q3: Is it Verified Vendor or Verified NGO?

* BOTH are verified, but for different safety purposes!
* Verified Vendors (Food Safety): Vendors must enter a valid 14-digit FSSAI Code. Your system decodes and verifies their license to ensure food safety.
* Verified NGOs (Targeted Alerts): When vendors post surplus food, server.js sends real-time email alerts ONLY to verified NGOs (isVerified = 1) who activated their account. This prevents spammers from taking free community meals.

Q4: What is the Dual-Engine Email Dispatcher?

* Engine 1 (Google Apps Script HTTPS Bridge - Primary): Cloud hosts like Render block SMTP ports (587/465). mailer.js calls a Google Apps Script HTTPS URL on standard Web Port 443 to send emails smoothly.
* Engine 2 (Nodemailer SMTP - Fallback): When testing locally on a laptop, mailer.js uses standard Gmail SMTP.
* Combined Result: 100% reliable email delivery on both cloud and local environments!